Gestion de rappels

Exercice de synthèse 2

Consignes générales :

- ⇒ **L'IDE à utiliser est PyCharm.**
- ⇒ Il est fortement recommandé d'ouvrir les supports de cours Python pour vérifier les définitions et les syntaxes.
- ⇒ Toutes les fonctions sont à écrire dans un fichier séparés appelé **functions.py**
- ⇒ Tous les tests des fonctions seront à faire dans le fichier **main.py**
- ⇒ Les instructions **print()** et **input()** sont interdites dans les fonctions sauf si le rôle même de la fonction est de permettre l'affichage ou la saisie d'informations utilisateur.

Acquis d'apprentissage :

A la fin de ce TP, chaque étudiant doit être capable de s'autoévaluer et d'identifier les notions qu'il maîtrise en Python et celles qu'il doit travailler davantage.

Présentation du problème

Vous êtes chargé de développer un **système de gestion de rappels** permettant à plusieurs utilisateurs de gérer leurs tâches quotidiennes. Ce système devra être interactif et offrir plusieurs fonctionnalités pour organiser efficacement les rappels.

Objectifs :

1. Manipuler les dictionnaires et les listes pour structurer les données des rappels et des utilisateurs.
2. Développer plusieurs fonctions modulaires pour chaque fonctionnalité du système.
3. Implémenter un système interactif via le terminal pour tester et démontrer le fonctionnement complet du programme.

Partie 1

Contraintes :

1. Chaque utilisateur peut avoir une liste illimitée de rappels.
2. La date des rappels doit être saisie sous le format **JJ- MM-AAAA** (Jour-Mois-Année) et validée. Si le format est incorrect, refusez l'entrée.
3. Le programme doit utiliser des dictionnaires pour structurer les données, et chaque rappel doit inclure :
   - ***rappel*** : La description du rappel (chaîne de caractères).
   - ***date*** : La date du rappel (chaîne de caractères).
   - ***priorité*** : Une priorité parmi "haute", "moyenne", "basse".

- ***complété*** : Un statut booléen indiquant si le rappel est complété.

Les fonctionnalités demandées :

1. **Ajout de rappels :**
   - Implémentez une fonction **est_entier(chaine)** qui prend en paramètre une chaîne de caractères et détermine si cette chaîne peut être interprétée comme un nombre entier positif. La fonction doit retourner *True* si la chaîne est composée uniquement de chiffres de 0 à 9, et *False* sinon.
   - Implémenter une fonction **valider_date(date)** qui prend en paramètre une chaîne représentant une date et qui vérifie si elle respecte le format "JJ-MM-AAAA". La fonction devra :
     - Vérifier que la date respecte le format attendu (longueur de 10 caractères avec des tirets aux bonnes positions).
     - Vérifier que les parties jour, mois et année sont toutes des entiers.
     - Vérifier que le mois est compris entre 1 et 12 et que le jour est un nombre valide pour ce mois particulier.
     - Prendre en compte les années bissextiles pour le mois de février (29 jours).
     - La fonction doit retourner *True* si la date est valide, sinon *False*.
   - Implémentez une fonction **ajouter_rappel(rappels, utilisateur, rappel, date, priorite)** qui permet :
     - D'ajouter un rappel sous forme de dictionnaire avec une **date**, une **priorité** (par défaut "moyenne") et un statut de complétion ("complété": False).
     - Si l'utilisateur n'existe pas dans le système, créez une nouvelle entrée pour cet utilisateur.
2. **Affichage des rappels :**

   Implémentez une fonction **afficher_rappels(rappels, utilisateur, tri_par)** pour afficher les rappels :
   - Triés par **date** ou **priorité**.
   - Ajoutez une icône ✅ pour les rappels complétés et ⏳ pour ceux encore en attente.
   - Si l'utilisateur n'a aucun rappel, affichez un message informatif.

Pour le tri, vous pouvez implémenter une fonction utilisant l'un des algorithmes de tri suivants :

```
/* Algorithme de tri par insertion */
Algorithme : tri_par_insertion
Variables :
      tab[100] : Tableau d'entiers
      i, k, tmp : entier
Début :
      (* Le tableau tab a été rempli par une saisie clavier *)
      pour i ← 2 à N faire
            tmp <- tab[i]
            k <- i
             tant que (k > 1 et tab[k - 1] > tmp) faire
                        tab[k] <- tab[k - 1]
                        k <- k - 1
            fin tant que
            tab[k] <- tmp
      fin pour
fin
```

Tri par insertion : Cet algorithme consiste à construire progressivement une liste triée en insérant chaque élément de la liste d'origine à sa position correcte dans la nouvelle liste triée, comme on le ferait avec un jeu de cartes. Il est efficace pour les petites listes ou les listes presque triées.

```
/* Algorithme de tri par sélection */
Algorithme : tri_par_selection
Variables :
      tab[100] : Tableau d'entiers
      i, k, min, tmp : entier
Début :
      (* Le tableau tab a été rempli par une saisie clavier *)
      pour i ← 1 à N-1  faire
      (* recherche l'indice du minimum *)
            min <- i
            pour k ← i+1 à N faire
                        si (tab[k] < tab[min]) alors
                              min <- k
                        fin si
             fin pour
       (* échange des valeurs entre la case courante et le minimum *)
            tmp <- tab[i]
            tab[i] <- tab[min]
```

```
            tab[min] <- tmp
        fin pour
fin
```

Tri par sélection : Cet algorithme identifie à chaque itération l'élément le plus petit (ou le plus grand, selon l'ordre de tri souhaité) et le place à sa position finale dans la liste, réduisant ainsi progressivement la portion non triée.

3. **Suppression de rappels :**

   Implémentez deux fonctions sans recourir à des fonctions prédéfinies telles que del, remove, pop, etc. :

   - **supprimer_rappel(rappels, utilisateur, rappel)** : Supprime un rappel spécifique.
   - **supprimer_tous_rappels(rappels, utilisateur)** : Supprime tous les rappels d'un utilisateur.

4. **Recherche de rappels :**

   Créez une fonction **chercher_rappel(rappels, utilisateur, mot_cle)** pour rechercher les rappels d'un utilisateur contenant un mot-clé spécifique.

   Vous avez la possibilité d'utiliser la fonction lower().

5. **Marquage comme complété :**

   Implémentez une fonction **marquer_complet(rappels, utilisateur, rappel)** qui marque un rappel spécifique comme complété.

6. **Sauvegarde et chargement :**

   Implémentez deux fonctions :

   - **sauvegarder_rappels(rappels, fichier)** : Sauvegarde les rappels dans un fichier texte.
   - **charger_rappels(fichier)** : Charge les rappels depuis un fichier texte (option avancée : utilisez JSON).

Partie 2

Ajoutez un menu interactif dans une méthode main() pour tester les fonctionnalités. Par exemple :

```
=== SYSTÈME DE GESTION DE RAPPELS ===

1. Ajouter un rappel

2. Afficher les rappels

3. Supprimer un rappel
```

4. Supprimer tous les rappels

5. Rechercher un rappel

6. Marquer un rappel comme complété

7. Sauvegarder les rappels

8. Charger les rappels

9. Quitter

Exemple d'utilisation :

- ***Ajoutez un rappel pour l'utilisateur "Alice" :***

Entrer le rappel : "Appeler le médecin"

Entrer la date (JJ-MM-AAAA) : 10-12-2024

Priorité (haute, moyenne, basse) : haute

- ***Affichez les rappels pour "Alice" triés par date :***

=== Rappels pour Alice ===

2024-12-10 haute | Appeler le médecin ⏳

- ***Marquez ce rappel comme complété et affichez les rappels mis à jour :***

=== Rappels pour Alice ===

2024-12-10 | haute | Appeler le médecin ✅